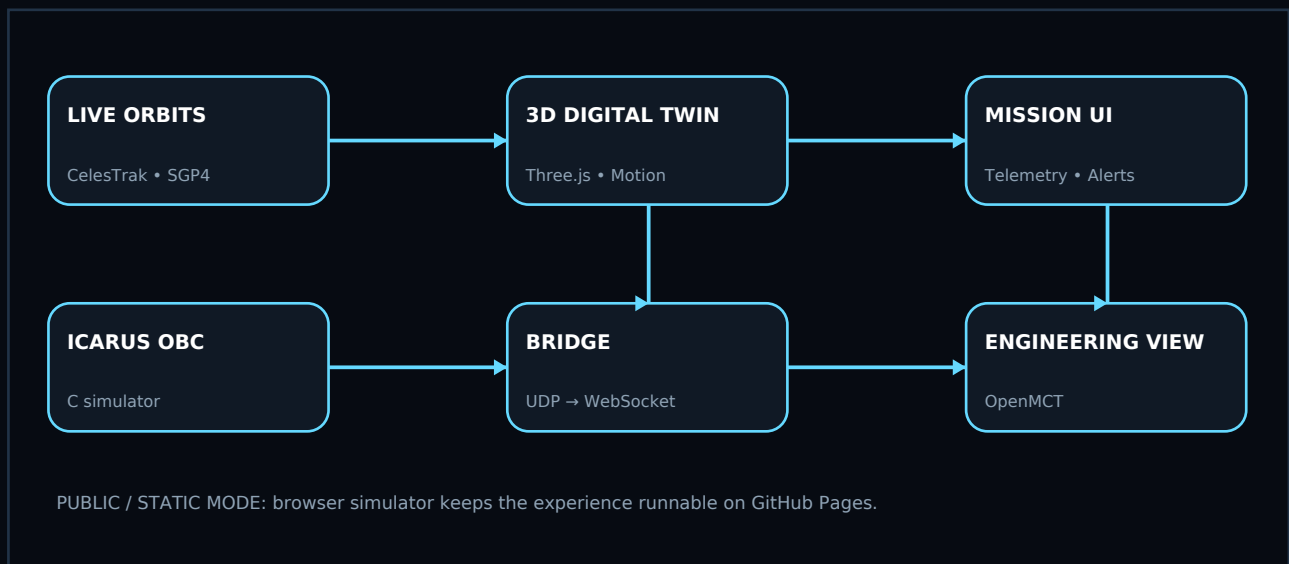


CubeSat Digital Twin + Icarus FSW

Merged engineering build manual — live orbital visualization, interactive 1U simulation, simulated onboard flight software, telemetry bridge and optional OpenMCT.



Design principle: build one coherent spacecraft story, not two competing dashboards. The 3D digital twin is the exhibition surface; Icarus FSW is the simulated onboard source of truth for telemetry and fault behavior.

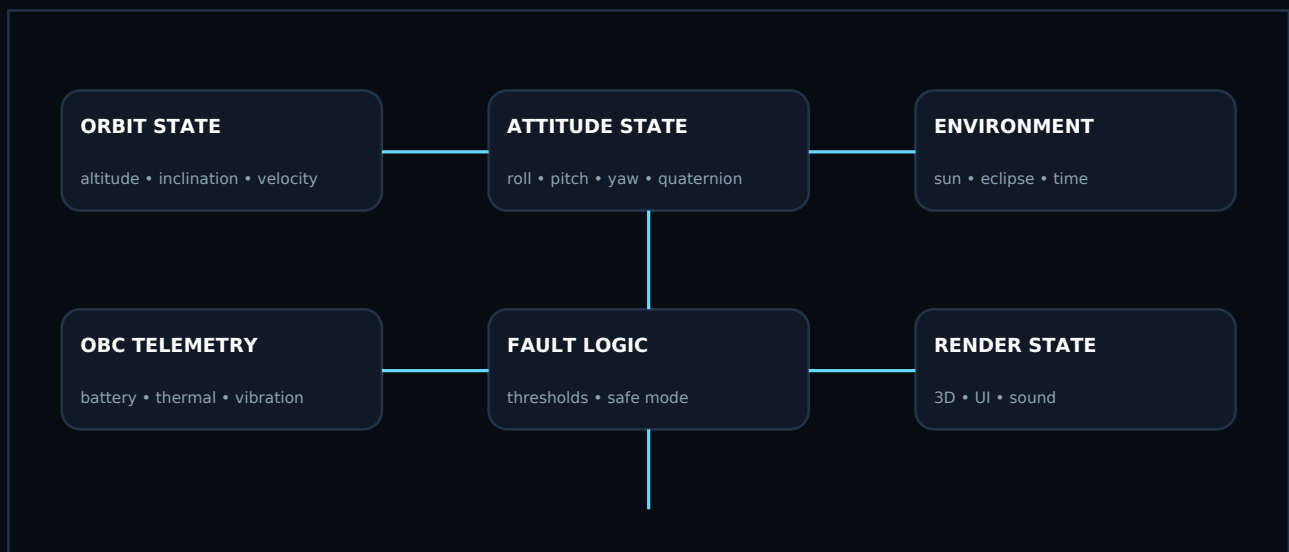
01 • System contract

The merged project has one primary visual product and one engineering telemetry path. The primary product is a 3D digital twin. Icarus FSW is a simulated onboard computer that produces spacecraft-state telemetry. The bridge converts the C process's UDP packets into WebSocket messages. The browser consumes a normalized telemetry schema. OpenMCT can subscribe to the same stream for engineering validation.

The most important rule is that the browser UI must not invent values independently of the telemetry/state model. A displayed battery value, thermal value, vibration value or safe-mode state must have one authoritative source.

The live orbital layer is separate from Icarus. Real spacecraft are propagated from current orbital elements. The simulated 1U spacecraft has its own controlled state. They can share the same mission clock but must not be confused as the same physical object.

The application must support two modes without changing the front-end components: LOCAL-FSW and BROWSER-FSW. LOCAL-FSW consumes the Icarus pipeline; BROWSER-FSW runs a deterministic TypeScript equivalent for public GitHub Pages and for development when the C/Python processes are not running.



Acceptance: a user can start the front-end with no backend and see a complete experience; a local user can additionally start the C/Python pipeline and see the same UI driven by external telemetry.

02 • Product architecture and runtime modes

Runtime A — public/static: Vite builds the site into static files. The browser obtains live orbital elements when the network source is available and runs the browser telemetry simulator. This mode is deployable to GitHub Pages.

Runtime B — exhibition/local: the browser runs the same 3D front-end but connects to a local WebSocket bridge. The C OBC sends UDP packets to Python; Python validates and forwards JSON to the browser. This mode is the engineering demonstration.

Runtime C — engineering: OpenMCT runs locally and subscribes to the bridge. It is not required for the main visual screen.

Mode	3D Twin	Real orbit data	Icarus C	OpenMCT
Public	Yes	Yes + fallback	No; browser simulator	Optional
Exhibition	Yes	Yes + fallback	Yes	Optional
Debug	Yes	Yes	Yes	Yes

Do not make the front-end wait for the bridge. The scene should render immediately, show a small telemetry source status, and switch to LOCAL-FSW only after the WebSocket connection is established and a valid packet is received.

03 • Repository layout

```
cubesat-digital-twin/ ■■■ src/ ■■■ main.ts ■■■ state/ ■■■ store.ts ■■■ types.ts ■■■ selectors.ts ■■■ orbit/
■■■ celestrak.ts ■■■ fallback.ts ■■■ propagate.ts ■■■ frames.ts ■■■ eclipse.ts ■■■ twin/ ■■■
earth.ts ■■■ atmosphere.ts ■■■ sun.ts ■■■ stars.ts ■■■ cubesat.ts ■■■ orbits.ts ■■■ camera.ts ■■■
fsw/ ■■■ browserFsw.ts ■■■ websocket.ts ■■■ telemetry.ts ■■■ adcs/ ■■■ attitude.ts ■■■ controller.ts ■■■
■■■ ui/ ■■■ hud.ts ■■■ profile.ts ■■■ controls.ts ■■■ alerts.ts ■■■ audio/ ■■■ synth.ts ■■■ styles/
■■■ tokens.css ■■■ app.css ■■■ fsw/ ■■■ src/ ■■■ main.c ■■■ telemetry.c ■■■ power.c ■■■ thermal.c ■■■
■■■ vibration.c ■■■ anomaly.c ■■■ Makefile ■■■ bridge/ ■■■ bridge.py ■■■ requirements.txt ■■■ openmct/ ■■■
dictionary-plugin.js ■■■ realtime-telemetry-plugin.js ■■■ public/ ■■■ fallback/ ■■■ orbital-data.json ■■■
.github/workflows/deploy.yml ■■■ package.json ■■■ vite.config.ts
```

Keep the C and Python components outside the Vite source tree. They are development/exhibition processes, not browser assets.

The browser package must not attempt to execute the C binary. That separation is essential for GitHub Pages.

04 • Frontend bootstrap

Use Vite + TypeScript. Keep the initial dependency list small: three, satellite.js and motion. A static build should be the only production artifact.

```
npm create vite@latest cubesat-digital-twin -- --template vanilla-ts cd cubesat-digital-twin npm install three satellite.js motion npm run dev npm run build npm run preview
```

Vite's current deployment guidance uses the dist output for static deployment. For a repository-page deployment, set base to '/REPOSITORY/'. GitHub Pages can be configured to deploy through GitHub Actions. [\[cite\]turn0search0](#)

```
// vite.config.ts import { defineConfig } from "vite"; export default defineConfig({ base: "/cubesat-digital-twin/" });
```

The first implementation milestone is a working Three.js canvas with resize handling, a camera and a render loop. Do not build telemetry cards before the scene is stable.

05 • 3D Earth and visual scene

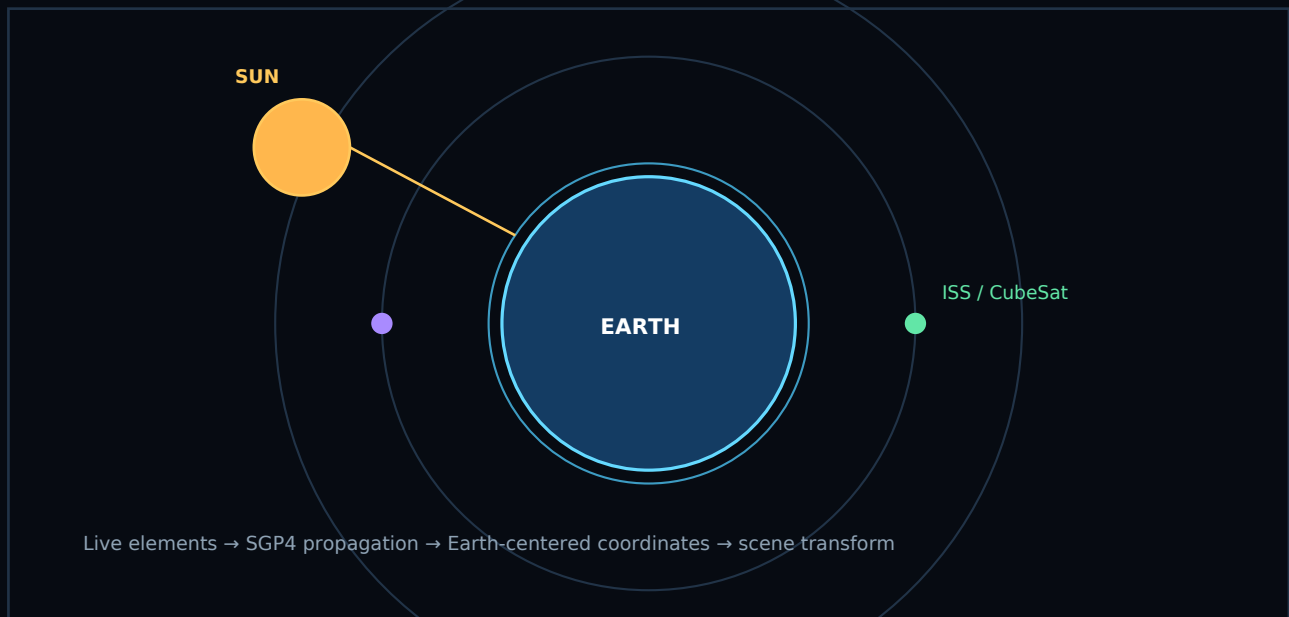
Layer order: star field → Sun → Earth → atmosphere → optional cloud shell → orbit paths → spacecraft markers → selected-object effects → UI overlay.

Use a high-quality Earth texture only if its redistribution terms are compatible with the repository. Otherwise use a procedural material or a clearly licensed asset. Do not make the project dependent on a remote texture URL at presentation time.

The atmosphere should be a second slightly larger sphere with a transparent additive-looking material. The visual objective is a thin blue rim, not a thick glowing shell.

The Sun should provide directional illumination. Add a separate warm sphere for the visible Sun. Keep the light physically directional enough that the Earth has a readable day/night boundary.

Stars should be points or instanced sprites, not thousands of individual meshes. Keep their count bounded.



Three.js WebGLRenderer uses WebGL 2 and exposes renderer capabilities and draw-call/triangle statistics useful during optimization. Keep shader error checking enabled during development. [\[cite\]turn0search1](#)

06 • Live orbital data

CelesTrak provides GP data through standard queries and supports JSON and OMM formats. The current documentation explicitly provides catalog-number, group and name queries. Use JSON OMM for the curated object set because it is structured and easy to validate. [\[cite\]turn0search4](#)

Use the ISS catalog number 25544 plus a curated list of CubeSat/LEO objects. The final object list can be changed without changing the renderer.

```
const LIVE_OBJECTS = [ { name: "ISS", catnr: 25544, kind: "station" }, { name: "CUBESAT-A", catnr: /* chosen at build time */ , kind: "cubesat" }, // 8-14 additional curated LEO objects ];
```

For each object, fetch current elements once at startup and only refresh on a coarse cadence. Never call CelesTrak from the render loop. Cache the normalized elements in memory for the session.

If a network request fails, use `public/fallback/orbital-data.json`. Mark the source as FALLBACK rather than hiding the failure.

Do not claim that fallback data is live. The UI should say LIVE or FALLBACK.

07 • SGP4 propagation and coordinate handling

satellite.js provides SGP4/SDP4 propagation and coordinate utilities. Its documentation supports TLE and OMM input and recommends OMM for new applications. [\[cite\]turn0search4](#)

```
import * as satellite from "satellite.js"; const satrec = satellite.json2satrec(ommRecord); function propagate(satrec, date) { const result = satellite.propagate(satrec, date); if (!result.position || !result.velocity) return null; const gmst = satellite.gstime(date); const geodetic = satellite.eciToGeodetic(result.position, gmst); return { positionEciKm: result.position, velocityEciKmS: result.velocity, latitudeDeg: satellite.degreesLat(geodetic.latitude), longitudeDeg: satellite.degreesLong(geodetic.longitude), altitudeKm: geodetic.height }; }
```

Keep physical state in kilometers, kilometers/second and UTC timestamps. Convert to Three.js scene units only at the renderer boundary.

Propagate the curated objects continuously. If CPU load becomes visible, propagate at a lower cadence and interpolate visual position between samples. The orbital data source is not a per-frame service.

08 • Earth-centered scene transform

Do not use Three.js coordinates as the authoritative orbital coordinate system. Use a dedicated transform function. A practical scale is 1 scene unit = 1000 km.

```
const EARTH_RADIUS_KM = 6371; const KM_PER_SCENE_UNIT = 1000; function sceneVectorFromKm(xKm, yKm, zKm) { return new THREE.Vector3( xKm / KM_PER_SCENE_UNIT, yKm / KM_PER_SCENE_UNIT, zKm / KM_PER_SCENE_UNIT ); }
```

Choose a consistent axis convention and document it. For example, X = equatorial reference axis, Y = equatorial orthogonal axis, Z = north. Apply one coordinate conversion into Three.js coordinates and never scatter sign flips throughout the code.

Ground tracks can be drawn as a separate visualization derived from latitude/longitude, but they are not required for the main exhibition.

09 • 1U CubeSat model

The 1U spacecraft is a visual model, not a CAD reproduction. Build it procedurally from a central body, rails, solar panels, face panels, a small antenna and an internal reaction-wheel disk.

The spacecraft should read clearly at medium distance. Use metallic grey, near-black solar cells and restrained cyan/white details. Avoid excessive emissive neon.

Hierarchy: CubeSatRoot → Body → SolarPanels → Antenna → ReactionWheel. Apply attitude to CubeSatRoot. Apply local animation to the reaction wheel and antenna.

During normal operation, keep the spacecraft opaque. During the ADCS recovery sequence, a short transparent/cutaway state can reveal the reaction wheel. This is the one internal visual explanation worth adding.

10 • Simulator orbit model

The simulator uses representative LEO values, not the team's actual mission. Default: near-circular orbit, 500 km altitude, 51.6° inclination. Eccentricity remains fixed at zero in the first version.

For circular orbit: $r = R_e + h$; $v = \sqrt{\mu/r}$; $T = 2\pi\sqrt{r^3/\mu}$. Use $\mu = 398600.4418 \text{ km}^3/\text{s}^2$ and $R_e = 6371 \text{ km}$.

```
function orbitFromAltitude(altKm: number) { const r = 6371 + altKm; const velocity = Math.sqrt(398600.4418 / r); const periodMin = 2 * Math.PI * Math.sqrt((r ** 3) / 398600.4418) / 60; return { radiusKm: r, velocityKmS: velocity, periodMin }; }
```

Altitude changes must update the orbit radius, velocity, period, camera framing and telemetry. The UI must not display hard-coded values.

Inclination changes the orbital plane. The simplest correct model for a circular educational orbit is to generate a ring in the orbital plane and rotate the ring by the inclination.

11 • Attitude and quaternion state

Expose roll, pitch and yaw because they are intuitive. Internally use a quaternion for orientation. This avoids making the renderer dependent on chained Euler rotations.

```
interface AttitudeState { rollDeg: number; pitchDeg: number; yawDeg: number; quaternion: THREE.Quaternion;  
angularVelocityDegS: THREE.Vector3; }
```

Convert the UI angles to a quaternion in a documented rotation order. Apply the quaternion to CubeSatRoot. Keep the displayed Euler values as the user-facing representation.

Do not let attitude affect the orbital path. Attitude is spacecraft orientation; orbit is spacecraft translational state. The two interact only through subsystems such as solar generation and ADCS, not through arbitrary visual coupling.

12 • ADCS disturbance and recovery

The disturbance sequence is deliberately simplified but closed-loop. At trigger time, apply a bounded angular velocity impulse. The controller measures attitude error against the target quaternion and applies a damped corrective response.

A practical visual controller is PD-like: $\text{torque} = -K_p \cdot \text{error} - K_d \cdot \text{angularVelocity}$, with saturation. The model is not flight-ready; it is an engineering demonstration of feedback.

```
const KP = 0.9; const KD = 0.35; const MAX_TORQUE = 2.0; const torque = clampVector( errorVector.multiplyScalar(-KP)
.add(omega.multiplyScalar(-KD)), MAX_TORQUE );
```

State sequence: NOMINAL → DISTURBANCE → ADCS_ACTIVE → ERROR_DECREASING → NOMINAL. Motion controls the presentation sequence; the underlying attitude state is still updated by the simulation.

Show a reaction-wheel indicator during ADCS recovery. Do not turn the entire spacecraft into a complicated mechanical animation.

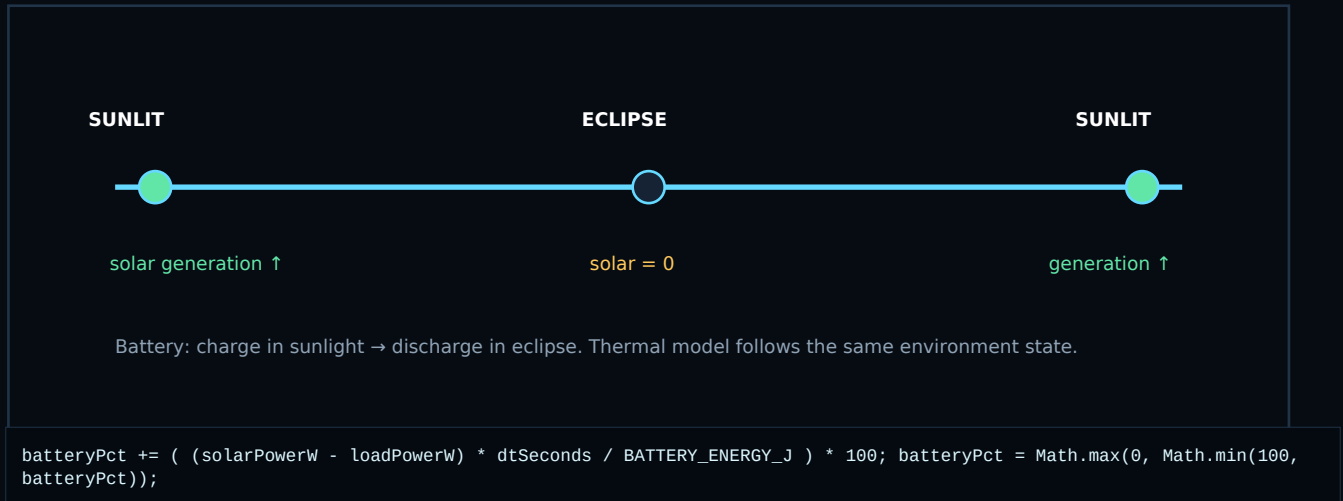
13 • Sunlight, eclipse and power

The original Icarus plan uses a 90-minute simulated orbit with 60 minutes sunlit and 30 minutes eclipse.

□filecite□turn0file0□L14-L19□ In the merged model, the browser should calculate eclipse from geometry for the 3D simulator, while the C OBC can use the same mission-clock state for its simplified power model.

Use a geometric Earth-shadow approximation: determine the Sun direction and satellite position in an Earth-centered frame; if the satellite is behind Earth and within the umbra criterion, mark ECLIPSE.

Solar input is positive in sunlight and near zero in eclipse. Battery state integrates generation minus load. Thermal state can trend upward in sunlight and downward in eclipse, with bounded noise.



Do not use a manual 'eclipse' toggle in the final exhibit. The visual should prove that the state follows the orbit.

14 • Icarus C OBC simulator

The Icarus source brief defines the C program as the replacement for the Python dummy generator and calls for a simulated LEO sunlit/eclipse cycle with telemetry sent by UDP. [filecite\[turn0file0\]L14-L19](#)

Keep the C engine intentionally small. Its job is to generate deterministic telemetry and fault states, not to become a full flight-software framework.

```
typedef struct { double timestamp_s; double altitude_km; double velocity_km_s; double roll_deg; double pitch_deg; double yaw_deg; double battery_pct; double solar_w; double temperature_c; double vibration_g; int eclipse; int safe_mode; int anomaly; } TelemetryPacket;
```

Split the model into modules: power.c, thermal.c, vibration.c, anomaly.c and telemetry.c. main.c owns the simulation clock and calls update functions at a fixed cadence.

Use a deterministic simulation timestep, for example 10 Hz, but transmit telemetry at a lower rate such as 5–10 Hz. The browser can interpolate or display the latest sample.

15 • C telemetry packet format

Use JSON for interoperability and debugging. UDP packets are small enough for this demonstration. The Python bridge should reject malformed packets rather than forwarding them.

```
{ "timestamp": 1757940000.25, "source": "ICARUS-OBC", "satellite": "CUBESAT-01", "orbit": { "altitude_km": 500.0,
"velocity_km_s": 7.61, "inclination_deg": 51.6 }, "attitude": { "roll_deg": 0.2, "pitch_deg": -0.4, "yaw_deg": 3.1 },
"power": { "battery_pct": 82.4, "solar_w": 1.18 }, "thermal": { "temperature_c": 28.4 }, "vibration": { "g": 0.12 },
"environment": { "eclipse": false }, "state": "NOMINAL" }
```

Keep field names stable. The browser and OpenMCT should consume this schema rather than C-specific structures. If a field is unavailable, use null and preserve the packet shape.

Version the schema if it changes: telemetry_schema_version: 1.

16 • Python UDP → WebSocket bridge

The bridge listens on UDP, validates and normalizes incoming JSON, then broadcasts each accepted packet to all connected WebSocket clients. This supports both the custom digital twin and OpenMCT.

```
# bridge.py – conceptual structure
import asyncio, json, socket
import websockets
CLIENTS = set()

def validate(packet):
    required = ["timestamp", "satellite", "power", "thermal", "vibration", "state"]
    return all(k in packet for k in required)

async def broadcast(message):
    dead = []
    for ws in CLIENTS:
        try:
            await ws.send(message)
        except Exception:
            dead.append(ws)
    for ws in dead:
        CLIENTS.discard(ws)

# UDP receive loop:
# 1. recvfrom()
# 2. decode JSON
# 3. validate
# 4. normalize
# 5. broadcast to WebSocket clients
```

Use localhost ports by convention, e.g. UDP 5005 and WebSocket 8082. Make both configurable through environment variables or command-line arguments.

The bridge should expose a health log: packet count, last packet timestamp, client count and malformed packet count. This is invaluable during exhibition debugging.

17 • Browser FSW fallback

GitHub Pages cannot run the C OBC or Python bridge. The browser therefore needs an Icarus-compatible simulator that produces the same normalized schema.

This is not a second visual system. It is a software substitute for the backend. The same telemetry renderer consumes both LOCAL-FSW and BROWSER-FSW packets.

```
interface TelemetrySource { start(onPacket: (p: TelemetryPacket) => void): void; stop(): void; mode: "LOCAL-FSW" | "BROWSER-FSW"; }
```

Browser-FSW should reproduce the important relationships: sunlit/eclipsed state, battery charging/draining, thermal trend, vibration baseline, and anomaly injection. Use deterministic equations and bounded noise so the demo is repeatable.

The UI should show a small source indicator such as 'FSW LINK • LOCAL' or 'FSW LINK • SIMULATED'. This prevents the public GitHub demo from being mistaken for the live C process.

18 • OpenMCT integration

OpenMCT remains an engineering/debug view. NASA's Open MCT tutorial demonstrates telemetry adapters for both historical requests and real-time WebSocket subscriptions. The realtime tutorial registers a provider with `supportsSubscribe` and `subscribe`, receiving telemetry through a WebSocket. [\[cite\]turn1search0](#)

Create a small dictionary plugin defining the telemetry points: Battery, Temperature, Vibration, Solar Power, Altitude, Roll, Pitch, Yaw and State. Create a realtime telemetry plugin that subscribes to the same WebSocket bridge.

```
// conceptual OpenMCT realtime provider
const provider = {
  supportsSubscribe(domainObject) {
    return domainObject.type === "cubesat.telemetry";
  },
  subscribe(domainObject, callback) {
    // subscribe to bridge stream // route point by telemetry key
    return () => { // unsubscribe };
  }
};
openmct.telemetry.addProvider(provider);
```

The Icarus source brief specifically requires three live plots and an alert condition widget.

[\[filecite\]turn0file0L27-L32](#) Configure these in OpenMCT for engineering validation, but do not place OpenMCT's UI on top of the main cinematic projector scene.

19 • Custom digital-twin telemetry UI

The main UI should show only the values that support the story. Recommended primary telemetry: Battery, Solar Input, Temperature, Vibration, Roll/Pitch/Yaw, Altitude, Velocity, Period and State.

Panel	Content
Object profile	Name, source, altitude, velocity, inclination, period
Flight state	NOMINAL / ADCS / SAFE MODE / ANOMALY
Power	Battery %, solar W, eclipse state
Thermal	Temperature °C, compact trend
Attitude	Roll/pitch/yaw and target error
Fault	Single active alert, timestamp and response

Do not recreate the Icarus reference's entire grid of plots. Its instrumentation language is useful, but the exhibition requires hierarchy and distance readability.

20 • Fault injection and safe mode

The original Icarus plan calls for keyboard-triggered thermal load spikes or battery drop to 0%, followed by a large critical anomaly and safe-mode response. [filecite]turn0file0[L20-L25] Keep one simple physical trigger in the final interface: SIMULATE ANOMALY.

Recommended behavior: thermal spike sets a high but bounded temperature; anomaly logic detects threshold breach; safe_mode becomes true; non-essential simulated loads reduce; battery drain changes; UI transitions to SAFE MODE; the 3D CubeSat can switch to a restrained safe visual state.

```
if (temperatureC > THERMAL_LIMIT || batteryPct <= BATTERY_CRITICAL) { anomaly = true; safeMode = true; } if (safeMode) {  
loadPowerW = SAFE_MODE_LOAD_W; adcsTarget = SAFE_ATTITUDE; }
```

Do not make the fault permanent. Provide a recovery/reset action or allow the demo to return to nominal after a fixed sequence. A presentation system must be repeatable.

21 • Motion animation and camera

Motion should animate the interface and selected Three.js objects, while the core physical simulation remains time-step driven. Motion documents animation of Three.js transforms and sequence-based coordination.

[\[cite\turn0search2\turn0search3\]](#)

Use Motion for: card entry, satellite selection, camera target transitions, telemetry emphasis, anomaly sequence, safe-mode banner, and reset. Use the Three.js render loop for continuous orbital movement.

```
import { animate } from "motion"; animate(profileCard, { opacity: [0, 1], y: [12, 0] }, { duration: 0.35, ease: "easeOut" });
```

Camera behavior: slow automatic orbit when idle; user drag/zoom when interacting; selection causes a smooth target transition; reset returns to a wide Earth view. The automatic camera should yield to user input rather than fight it.

Motion supports Three.js object animation and can be used in sequences with HTML/SVG.

[\[cite\turn0search2\turn0search6\]](#)

22 • Sound design

Use Web Audio API so the project has no mandatory audio-file dependency. Sound is polish, not a functional requirement.

Five cues are enough: interface click, satellite selection, telemetry pulse, anomaly warning, recovery confirmation. Audio must be disabled by default until the first user interaction if browser policy requires a gesture.

```
function tone(ctx, frequency, duration, gainValue) { const osc = ctx.createOscillator(); const gain = ctx.createGain();
osc.frequency.value = frequency; gain.gain.setValueAtTime(0.0001, ctx.currentTime); gain.gain.exponentialRampToValueAtTime(
gainValue, ctx.currentTime + 0.01 ); gain.gain.exponentialRampToValueAtTime( 0.0001, ctx.currentTime + duration );
osc.connect(gain).connect(ctx.destination); osc.start(); osc.stop(ctx.currentTime + duration); }
```

Keep sound subtle enough that a projector's speakers do not become distracting. The anomaly cue can be slightly stronger because it marks the narrative climax.

23 • Visual design specification

Reference DNA: the uploaded Digital Space Twin image shows a dark, immersive 3D space environment, a compact search/object panel, an object profile panel, orbital lines and small status controls. Translate this into a simpler exhibition layout rather than copying the density.

Use one dominant canvas. Top-left: small identity/status. Top-right: camera/sound controls. Lower-left: selected object. Lower-right: CubeSat telemetry and controls. Bottom: compact simulation time/status line.

Use 12–18 px corner radii, thin borders, dark translucent panels and large whitespace. Typography should be white with muted secondary labels. Cyan indicates active/selected. Green indicates nominal. Amber means attention. Red is reserved for critical anomaly.

The visual system should feel closer to a mission-control instrument than a conventional website. Avoid large headings, marketing copy, gradients everywhere, or excessive glass effects.

Projection rule: design for 16:9 at 1920×1080 first. Controls should be legible from several meters away. Avoid any critical information smaller than roughly 16 px in the projected UI.

24 • Performance and reliability

Three.js WebGLRenderer exposes renderer capabilities and statistics such as draw calls and triangles. Use `renderer.info` during development to identify expensive scene construction. Keep shader checking enabled during development. [\[cite\]turn0search1](#)

Performance order: stable 3D renderer → low object count → shared materials → bounded star count → capped pixel ratio → optional post-processing. Do not add bloom until the base scene is stable.

Cap device pixel ratio around 1.5–2.0 for the projector. If sustained frame rate falls below approximately 45 FPS, first reduce effects and orbit-line density rather than reducing UI quality.

Live data failures must not stop the render loop. Malformed orbital records are discarded individually. WebSocket disconnects should switch to BROWSER-FSW after a short grace period rather than blanking telemetry.

Audio failure must not throw. If WebGL initialization fails, show a readable diagnostic screen rather than an empty page.

25 • Testing and acceptance matrix

Test	Expected result
No network	Earth and fallback satellites still render
CelesTrak unavailable	UI shows FALLBACK; no crash
ISS selected	Camera frames ISS; profile updates
Altitude +100 km	Orbit grows; velocity falls; period rises
Inclination +20°	Orbital plane tilts smoothly
Roll +20°	CubeSat rotates about roll axis
Disturbance	Angular error rises then decreases
Eclipse	Solar input ≈ 0 and battery trend reverses
Thermal anomaly	State enters SAFE MODE
Battery anomaly	State enters SAFE MODE
UDP malformed packet	Bridge logs/rejects it
WebSocket disconnect	Front-end continues in browser fallback
OpenMCT subscriber	Telemetry points update in real time
GitHub Pages	Static build loads with repository base path

Unit test the orbital equations independently. Integration-test the telemetry packet path. Then run the full exhibition test on the actual presentation laptop and projector.

26 • GitHub Pages deployment

Vite's current static deployment guidance says to set the correct base path for a repository deployment, build to dist, and use GitHub Actions as the Pages source. [\[cite\]turn0search0](#)

```
// vite.config.ts export default defineConfig({ base: "/YOUR-REPOSITORY/" });  
# local verification npm ci npm run build npm run preview
```

Use a GitHub Actions workflow with checkout, Node setup, npm ci, npm run build, configure-pages, upload-pages-artifact and deploy-pages. The browser application is the only part deployed to Pages. C/Python/OpenMCT remain local engineering tools.

Repository assets must use Vite's base-aware paths. Never hard-code '/assets/...' if the site is deployed under '/repository/'. Use import.meta.env.BASE_URL or imported assets.

Public-mode behavior must be tested from the actual GitHub Pages URL because localhost hides path and cross-origin mistakes.

27 • Two-day build schedule and cut order

Time	Build	Cut if behind
0-2 h	Vite + Three.js + Earth + stars + camera	—
2-5 h	Live CelesTrak + SGP4 + ISS + 10-15 objects	Fancy effects
5-7 h	Orbit lines + selection + profile	Extra object metadata
7-10 h	1U CubeSat + altitude/inclination/attitude	Cutaway
10-13 h	Browser FSW + telemetry cards	Sound
13-16 h	C OBC + UDP + Python bridge	Extra telemetry channels
16-18 h	Connect WebSocket + OpenMCT	Historical telemetry
18-20 h	Eclipse + battery/thermal/vibration	Complex thermal physics
20-23 h	Anomaly + safe mode + ADCS sequence	Micro animations
23-26 h	Motion + camera polish + sound	Post-processing
26-28 h	Performance + projector testing	New features
28-30 h	GitHub Pages + README + rehearsal	Anything non-essential

If the C engine becomes unstable, the source brief's recommended fallback is the nominal telemetry loop plus a reliable anomaly trigger. [filecite\turn0file0\L42-L46](#) In this merged architecture, browser-FSW is an additional safety net.

Do not spend the final hours adding features. Spend them making the first 90 seconds flawless.

28 • Exhibition script and technical defense

Opening: show Earth and real orbital traffic. Select the ISS. State that its position is propagated locally from current orbital elements rather than being a pre-rendered animation.

Transition: select the simulated 1U CubeSat. Increase altitude. Point out the coupled changes: larger orbital radius, lower circular velocity and longer period. Change inclination and show the plane tilt.

Telemetry: point to battery, solar input, thermal and vibration. Explain that the visible values can be generated by the C-based Icarus OBC and transported through UDP/WebSocket.

Fault: trigger the anomaly. Show the temperature/battery threshold breach, safe-mode transition and UI response. If ADCS is included in the same sequence, let the attitude recover before returning to nominal.

Technical defense: the system is deliberately not flight software. SGP4 is used for visualization of public orbital elements; the C OBC is a deterministic telemetry/flight-software demonstrator; ADCS is a bounded educational control model. This distinction is important.

Questions to expect: Why SGP4? Why OMM? Why WebSocket? Why separate OpenMCT? Why can't GitHub Pages run the C process? How does eclipse affect battery? Why use quaternions? What would change for a flight-qualified implementation?

Answer to the GitHub Pages question: static hosting can run the browser simulation and real-orbit layer, but not a server process. Therefore the browser simulator implements the same telemetry contract as Icarus, while local exhibition mode can connect to the real C/Python path.

Final acceptance: the application must look complete even when no one touches it, but every interactive action must produce a technically coherent response. That balance is the central design requirement.